

# AI Coding Dictionary

## AI Hero

- AI Coding Dictionary
- Section 1 — The Model
  - AI
  - Model
  - Parameters
  - Training
  - Inference
  - Token
  - Next-token prediction
  - Non-determinism
  - Model provider
  - Harness
  - Model provider request
  - Input tokens
  - Output tokens
  - Prefix cache
  - Cache tokens
- Section 2 — Sessions, Context Windows & Turns
  - Stateless
  - Context
  - Context window
  - Stateful
  - Agent
  - System prompt
  - Session
  - Turn
- Section 3 — Tools & Environment
  - Environment
  - Filesystem
  - Tool
  - Tool call

- Tool result
  - MCP
  - Permission request
  - Permission mode
  - Agent mode
  - Sandbox
- Section 4 — Failure Modes
  - Sycophancy
  - Hallucination
  - Parametric knowledge
  - Knowledge cutoff
  - Contextual knowledge
  - Attention relationship
  - Attention budget
  - Attention degradation
  - Smart zone
- Section 5 — Handoffs
  - Clearing
  - Handoff
  - Primary source
  - Secondary source
  - Handoff artifact
  - Spec
  - Ticket
  - Compaction
  - Autocompact
- Section 6 — Memory and Steering
  - Memory system
  - AGENTS.md
  - Progressive disclosure
  - Context pointer
  - Skill
  - Subagent
- Section 7 — Patterns of Work
  - Human-in-the-loop

- AFK
- Automated check
- Automated review
- Human review
- Vibe coding
- Design concept
- Grilling
- Prototyping
- DX
- AX

# AI Coding Dictionary

**AI coding can feel like it's just for experts.** Unexplained jargon. Mysterious failures. Bills that don't seem to match the work.

It isn't, really. A lot of the confusion is manufactured: **there's a whole VC-funded economy that benefits from keeping it hard to understand.**

The basic terms of engagement are learnable in an afternoon. Once you have them, the whole thing stops feeling like guesswork.

Why does context degrade? Why is the bill so high? Why does the same prompt behave differently from one day to the next?

Each has a clean answer, once someone tells you the words to use.

That's what this dictionary is for. **The vocabulary of AI coding, translated into plain English.**

Want more than the vocabulary? Join 62,000+ developers at [aihero.dev/newsletter](https://aihero.dev/newsletter).

# Section 1 — The Model

## AI

A moving label, not a technology. "AI" doesn't name a fixed thing the way model or token does — it points at whatever computers can newly, impressively do. Right now it points at large language models. It has pointed at very different things before:

| Era       | What "AI" meant                                                                                       |
|-----------|-------------------------------------------------------------------------------------------------------|
| 1950s     | Symbolic reasoning — theorem provers, checkers programs.                                              |
| 1960s–70s | Rule-based symbolic programs — ELIZA, SHRDLU.                                                         |
| 1980s     | Expert systems — thousands of hand-written if-then rules encoding human expertise.                    |
| 1990s     | Game-tree search — Deep Blue beating Kasparov (1997). Researchers avoided the word "AI" entirely      |
| 2000s     | Statistical machine learning — spam filters, recommenders. Still sold as "machine learning", not "AI" |
| 2010s     | Deep learning — image recognition (AlexNet, 2012), AlphaGo (2016).                                    |
| 2020s     | Large language models — ChatGPT (2022) made "AI" mean chatbots                                        |

The pointer moves by a known mechanism, sometimes called the AI effect: once a technique works reliably, it gets renamed — it's "just" search, "just" statistics — and "AI" slides forward to the next unsolved thing. The observation is old. Bertram Raphael put it this way in 1971: "AI is a collective name for problems which we do not yet know how to solve properly by computer." Larry Tesler's version, from around 1979: "Intelligence is whatever machines haven't done yet."

This is why conversations about AI so often talk past each other. A claim like "AI can't reason" or "AI is overhyped" carries a hidden timestamp — it may be about expert systems, about 2010s image classifiers, or about last month's LLM, and each reference supports a different conclusion. When a discussion about AI stalls, the fix is

usually to swap the word for whichever precise term is actually meant: the model, the harness, the agent, the context it was given.

*Avoid:* "AI" in any technical claim — name the part you mean instead. "AI coding" as a label for the practice is fine; "the AI is hallucinating" is not.

*Usage:*

"The CTO wants to know whether AI could handle the triage queue."

"Translate that before scoping it — she means an LLM in a harness with access to the ticket system. 'AI' on its own isn't a spec."

## **Model**

The parameters. Stateless — does next-token prediction and nothing else. "Claude Opus 4.x" and "GPT-5.x" are models. On its own a model can't do anything agentic; it has to be harnessed.

Models can't read files, run commands, browse the web, or remember yesterday — it takes tokens in and predicts tokens out, once per model provider request. Everything that feels like an agent working — choosing tools, reading results, looping until the task is done — is the harness orchestrating many of those predictions in a row.

Model providers ship models in tiers: a large one that's smartest but slow and expensive, and smaller ones that are faster and cheaper but less capable. Picking a tier is a real decision — heavyweight for planning and hard debugging, lightweight for mechanical changes — and harnesses let you switch mid-session.

Being strict about the word also sharpens diagnosis. "The model is bad at this" is a specific claim — the same model in a different harness, or with a different context, often behaves completely differently. Before blaming the model, check what it was given: most disappointing output traces back to context or harness, not parameters.

*Usage:*

"Should we switch the model from Sonnet to Opus for the planning step?"

"Try it — but the harness is doing most of the lifting on this task. The model swap won't help if the system prompt and tools are wrong."

## **Parameters**

The numbers inside a model — often billions of them — tuned during training. Everything the model "knows" lives in them. Training sets them; inference uses them unchanged. Also called *weights*.

Mechanically, the parameters are what turn input into output. Next-token prediction is a giant calculation: the tokens in the context window go in, get multiplied through the parameters, and a prediction for the next token comes out. There is no database of facts inside the model, no code lookup table — just these numbers, arranged so that the calculation tends to produce useful output. Facts the model can recite from training, like a standard library API, are parametric knowledge: stored in the parameters, not retrieved from anywhere.

The detail worth internalising is that parameters are frozen after training. Nothing you do in a session changes them — no correction you make, no codebase you show it, no mistake it learns from. Every session runs on the same numbers. This is why the model is stateless, why its built-in knowledge stops at the knowledge cutoff, and why anything project-specific has to arrive via context instead. The only way parameters change is more training — which produces, in effect, a different model.

*Usage:*

"Can we fine-tune it on our codebase?"

"That'd update the parameters — different model afterwards. For one project it's almost always cheaper to load the codebase as context than to retrain."

## Training

The process that sets a model's parameters, by exposing it to vast amounts of text and adjusting parameters to improve next-token prediction. A one-time, expensive process done by the model provider. Encompasses both pre-training (the bulk run) and post-training (later refinements like instruction-following and safety); the distinction doesn't matter at this glossary's level.

The mechanism is repetition at scale: show the model a stretch of text, have it predict the next token, nudge the parameters toward whatever the actual next token was, and repeat across trillions of tokens. Nothing is stored as facts or rules — everything the model "knows" is a side effect of getting better at prediction, compressed into the parameters as parametric knowledge.

Two consequences matter day to day. Training ends at a point in time, so the model has a knowledge cutoff — it hasn't seen the library version you upgraded to last month. And training is not something you can do: when the model doesn't know your codebase, your conventions, or your internal APIs, the fix is never "teach the model" — it's putting that material into context, the one input you control.

*Usage:*

"Can we get it to know our internal API?"

"Not via training — that's a months-long process by the model provider. Load the API docs into context instead, that's the lever you actually have."

## Inference



Running a trained model to generate output — what happens on every model provider request. Parameters stay fixed; the model just does next-token prediction over the context it's given. Cheap relative to training, but billed per token and the dominant cost of using a model.

A model's life splits into two phases:

| Phase     | When it happens                  | What it does                                                    | Parameters    |
|-----------|----------------------------------|-----------------------------------------------------------------|---------------|
| Training  | Once, before release             | Produces the parameters from a training corpus                  | Being written |
| Inference | Every time anyone uses the model | Runs the frozen parameters over your context to generate tokens | Read-only     |

Nothing you do at inference time writes back to the parameters — that's the reason a correction you make today doesn't stick tomorrow. The model that makes the same mistake next session, after you carefully explained the fix, hasn't ignored you; it's incapable of learning from the exchange. The model is stateless — continuity has to come from outside it — from the context window or a memory system.

This mechanism also explains how you're billed. Every request runs the model over the full context, so cost scales with input tokens and output tokens, and an agent making dozens of tool calls pays for inference on each round trip. This is why context size is a cost question as well as a quality one.

*Usage:*

"Why does the bill scale with usage instead of being a flat license?"

"You're paying for inference — every model provider request runs the model on the provider's hardware. Training already happened, but inference costs accrue per request, and a single turn can expand into many requests when tools are called."

## Token

The atomic unit a model reads and writes. Roughly word-sized but not exactly — common words are one token, rare or long ones split into several. Context window size, cost, and latency are all counted in tokens.

Text becomes tokens via a tokenizer: a fixed vocabulary of tens of thousands of fragments, learned before training, that splits any input into a sequence of vocabulary entries. The model never sees characters or words — every piece of text is converted to tokens on the way in, and next-token prediction produces output one token at a time on the way out.

As a rule of thumb, a token is about three-quarters of an English word, so a thousand tokens is roughly 750 words. Code is less predictable: common keywords and idioms tokenize compactly, while generated identifiers, hashes, base64 blobs, and minified output split into many tokens per "word". The pattern: text that appeared often in the tokenizer's source material gets short, efficient encodings; text that didn't gets chopped into many small pieces. A hash like `a3f9c2e1` never appeared anywhere, so it splits into many tokens, while `function` is one. This is why a small-looking file full of unusual strings can occupy a surprising share of the context window.

Tokens are the unit everything else is measured in. Cost is per token — providers bill input tokens and output tokens separately. Speed is tokens per second, since output is generated one token at a time. And the context window is a fixed number of tokens, so the token count of your files decides how much fits.

*Avoid:* "word" — token boundaries don't match word boundaries, and tokens-per-second / tokens-per-dollar are the units that actually matter.

*Usage:*

"How big is this prompt going to be?"

"Run it through the tokenizer — the schema's compact but the JSON keys are weird, so they'll split into more tokens than you think."

## Next-token prediction

What the model actually does. Given a context, it samples one next token, appends it, and runs again. Every output — a sentence, a tool call, a thousand-line file — is built one token at a time. The model has no other mode of operation.

Each step works the same way: the tokens in the context window are run through the parameters, which produce a probability for every token in the vocabulary — this one is very likely next, that one less so. One token is sampled from those probabilities, appended, and the loop runs again with the slightly longer context. That sampling step is why the same prompt produces different output on different runs: non-determinism is built into the mechanism, not a bug layered on top.

Holding onto this mechanism explains behaviour that otherwise looks strange. The model never checks whether a token is *true* before emitting it — only whether it's *likely* — which is the root of hallucination. It commits to each token as it goes, so a confident-sounding opening sentence can steer the rest of the answer wrong. And because output tokens are produced strictly one at a time, generation speed puts a floor on how fast any agent can work.

*Usage:*

"How does the agent 'decide' to call a tool?"

"It doesn't — it's next-token prediction all the way down. The tool call is just a structured string the harness parses out of the output stream."

## Non-determinism

The same input can produce different output. Run a model twice with identical context and you may get two different answers — sometimes a word, sometimes a completely different approach. Nothing in your code has to change for this to happen.

It's a property of how models generate text, and how model providers serve requests. During inference, the model produces a probability distribution over possible next tokens and one is sampled from it — usually with some randomness on purpose, since always picking the most likely token produces repetitive, lower-quality text. One differently-sampled token early in a response changes every token after it, which is how a single different word becomes a completely different approach. Provider-side serving adds more variation on top: requests are batched together on shared hardware, and tiny floating-point differences between batches can tip a close call between two tokens. There's no setting you can flip to make it all go away.

Expect a spread of results from an agent on the same task. Most responses fall within a reasonable bell curve of quality — that's why the non-determinism is tolerable at all — but the tails are real: some days the model will feel sharp; some days it'll feel like it's lost the plot. Same task, different rolls of the dice. This has two practical consequences. Retrying is a legitimate strategy: a failed attempt is one draw from the distribution, and a fresh attempt at the same task may simply land better. And verification matters more than it would with deterministic tools — you can't test an agent's behaviour once and rely on it repeating, so automated checks have to catch the bad draws.

Be careful not to over-narrativize this. Humans are pattern-matching machines, and a string of bad runs can feel like proof that "the model got worse this week." Usually it's just the distribution.

*Usage:*

"Claude has been awful today. Did they ship a worse version?"

"Probably not — model output is non-deterministic. You're going to have good days and bad days on the same task. Try again tomorrow before you go looking for a cause."

## **Model provider**

Whatever serves a model for inference. Usually a remote service (Anthropic, OpenAI, Google), but can also be local — Ollama, LM Studio, llama.cpp running on your own machine. The harness doesn't run the model itself; it asks a provider to.

The provider owns the machinery: the parameters live on its hardware, and every model provider request is the harness sending tokens over the network and getting predictions back. That makes the provider the source of a whole category of problems that get misattributed to the model or the harness — rate limits, degraded capacity, and outages all live here. When the agent stalls mid-session or errors on every turn, the provider's status page is worth checking before anything else.

The provider also sets the commercial terms: per-token pricing for input and output tokens, prefix cache discounts, and which models are available at all. Note that the provider and the model's maker can be different companies — Bedrock, Vertex, and OpenRouter serve other people's models.

Local providers trade capability for control: the models that fit on your own hardware are far smaller than the frontier ones, but nothing leaves the machine and there's no bill per token.

*Usage:*

"Can we run this offline for the air-gapped client?"

"Swap the model provider to a local one — Ollama or llama.cpp on their box. The harness doesn't care, it just hits a different endpoint."

## **Harness**

Everything around the model that turns it into an agent: tools, system prompt, context-window management, permissions, hooks. **Claude.ai** and **Claude Code** run on the same model but behave differently because their harnesses differ.

The model itself only does one thing: take text in, produce text out. It can't read a file, run a command, or remember the last turn. The harness supplies all of that. It assembles the context for each model provider request, executes the tool calls the model asks for, feeds the tool results back in, stores the session history, asks you for permission before risky actions, and decides when to compact. The agent loop — model proposes, harness executes, repeat — is run by the harness.

This matters for diagnosis. When behaviour differs between two products, or between yesterday and today, the model is often not the variable — the harness is. A different system prompt, a different set of tools, a changed permission default, or a new context-management strategy all change behaviour without any change to the model. It also means the harness is where most of your configuration lives: AGENTS.md files, permission settings, and hooks are all instructions to the harness, not the model.

Examples: Claude Code, Cursor, Codex CLI — and Claude.ai, which is a chat harness rather than a coding one.

*Usage:*

"Same model, why is Claude Code editing files and Claude.ai just answering questions?"

"Different harnesses — Claude Code has filesystem tools, a different system prompt, and a permission layer. The model isn't the variable here."

## Model provider request

One round-trip from the harness to the model provider. The harness sends the current context; the provider returns one response (a tool

call or a final answer). A single user message can spawn many model provider requests if the agent calls tools — each tool result triggers another request.

Each request carries everything: the system prompt, the full conversation so far, every tool result. The model is stateless, so the provider keeps nothing between requests — request forty re-sends what request thirty-nine sent, plus one more tool result. The prefix cache exists to make this repetition affordable.

The request is also the unit of billing. Input tokens, output tokens, and cache discounts are all counted per request, which is why an innocuous-looking question can cost a surprising amount: the cost isn't proportional to your message, it's proportional to the number of requests times the size of the context each one carries.

It's worth keeping the request distinct from the turn. A turn is one exchange with you, and a single turn — "fix the failing test" — plays out as a chain of requests:

| Request | Model returns                     | Harness then                          |
|---------|-----------------------------------|---------------------------------------|
| 1       | Tool call: run the tests          | Runs them, appends the failure output |
| 2       | Tool call: read the test file     | Appends the file contents             |
| 3       | Tool call: read the source file   | Appends the file contents             |
| 4       | Tool call: edit the source file   | Applies the edit, appends the result  |
| 5       | Tool call: run the tests again    | Runs them, appends the pass output    |
| 6       | Final answer: "fixed, tests pass" | Shows it to you                       |

Six requests for one turn — each one re-sending the whole context. When you wonder where the tokens went, count the requests, not the turns.

*Usage:*

"One question burned forty thousand tokens?"

"Look at the tool calls — twelve grep, eight read, four edits. Each tool result spawns another model provider request, and the whole session prefix re-sends every time."

## **Input tokens**

Tokens the harness sends on each model provider request — the system prompt, the conversation history, tool results, everything the model reads before it writes. Billed at a lower rate than output tokens, because they are less expensive to process than output tokens.

When doing AI coding, input tokens make up most of your bill. The model is stateless, so each turn re-sends the entire session as input: your first message, every response, every tool result since. The input for turn fifty contains the previous forty-nine turns. A single model provider request might produce a few hundred output tokens but re-send a hundred thousand input tokens of accumulated history.

The prefix cache reduces the cost: history that exactly matches a previous request is billed as cheap cache tokens rather than full-price input. When input costs still hurt, the fix is to shrink what gets re-sent — clearing or compacting between tasks.

*Usage:*

"Bill's high but the agent's barely writing anything."

"It's the input tokens — every turn re-sends the whole session. Without the prefix cache you re-pay for the history each request."

## **Output tokens**

Tokens the model generates back. Billed at a higher rate than input tokens — commonly around five times the rate — since they cost more compute to produce.

Everything the model writes counts: the prose you read, the code it emits, tool calls, and any extended thinking the model does before



answering. That last one surprises people — reasoning tokens are billed as output even when the harness often doesn't show them to you.

Output tokens also set the pace of a session. The model reads input quickly but generates output one token at a time, so when a turn feels slow, it's almost always the output being written, not the input being read. A long wait usually means a long answer is coming.

*Usage:*

"The refactor session is burning through credit even though the inputs are small."

"Agent's rewriting whole files instead of patching. Output tokens cost roughly five times the input rate — get it emitting edits and the bill drops."

## **Prefix cache**

The provider-side store that lets consecutive model provider requests skip re-processing a shared prefix. When the start of a request matches the start of a recent one — same system prompt, same history up to some point — the provider reuses its prior work and bills those tokens as cache tokens at a much lower rate.

The cache pays off because sessions grow append-only. Every request re-sends the whole history as input tokens (see that entry for why), and in a normal session the history only changes at the end — each request is the previous one plus a few new messages. The provider processes the long shared beginning once, stores the result, and picks up from where the prefix ends. Without the cache, a 50-turn session would pay to re-process turn one fifty times.

Caches also expire. How long an entry stays warm varies per model provider — typically minutes, not hours. Leave a session idle past the window and the next request rebuilds the prefix at full price once before caching resumes. This is mostly a harness builder's

concern; as a user, the visible effect is that requests after a long pause cost more than the ones before it.

*Usage:*

"Why did the bill spike halfway through the session?"

"Harness started injecting the current time into the system prompt every turn. Prefix cache breaks at the first changed token, so every request after that billed at full rate."

## Cache tokens

Input tokens the provider has cached from a previous model provider request so it doesn't have to re-process them. When consecutive requests share a prefix, the provider reuses the work via its prefix cache and bills the cached portion at a much lower rate. The lever that makes long sessions affordable — without it, every turn re-pays for the whole history.

The reason this matters is how sessions are billed. The model is stateless, so every request resends the entire conversation — system prompt, every message, every tool result — as input tokens. By turn fifty, each request carries fifty turns of history, and you'd pay full rate on all of it, every time. The cache changes the maths: tokens the provider has already processed in an identical prefix are billed as cache tokens, often at a tenth of the input rate or less. On a long session, most of what you send is cache tokens, and the bill stays sane.

An example shows when tokens are cached and when they're not. Each letter stands for a block of conversation content; each request sends the conversation so far:

| Request sends | Cached  | Billed at full rate | Why                                      |
|---------------|---------|---------------------|------------------------------------------|
| AB            | nothing | AB                  | First request — nothing to match against |
| ABC           | AB      | C                   | AB is an exact prefix of the previous    |

| Request sends | Cached | Billed at full rate | Why                                           |
|---------------|--------|---------------------|-----------------------------------------------|
|               |        |                     | request                                       |
| ABCD          | ABC    | D                   | Prefix still intact                           |
| AXCD          | A      | XCD                 | An edit changed B to X; the match fails there |

The cache is fragile in a specific way: it matches exact prefixes. If anything changes earlier in the conversation — the harness reorders content, a timestamp updates, a file's representation shifts — the cache misses from that point onward and everything after it is billed at full input rate. Caches also expire after a few minutes of inactivity, so a session resumed after a long pause re-pays its history once. When a session's cost jumps without an obvious cause, compare cache tokens to input tokens in the usage report — a broken cache shows up there first.

*Usage:*

"Cost on long sessions is brutal — eight bucks for a refactor."

"Check the cache tokens. If the harness is reordering the system prompt or files between turns, the prefix breaks and you re-pay full input rate every request."

# Section 2 — Sessions, Context Windows & Turns

## Stateless

Carries no information forward. The model is stateless across model provider requests — each request resends the full context window, because the model has no way to see anything else. An agent is stateless across sessions by default: a new session starts empty, with no trace of prior ones. Counterpart to stateful.

The model itself is permanently stateless: its parameters are frozen after training, and nothing you do at inference changes them. The model doesn't learn from your corrections, doesn't remember being told the same thing yesterday, and isn't getting to know you — however much the conversation feels otherwise. The feeling of continuity within a session is manufactured by the harness, which keeps the transcript and re-sends it with every request. The model isn't remembering the conversation; it's re-reading it.

The practical consequence: if you want something remembered across sessions, you have to write it down somewhere the agent will read it back. That's what AGENTS.md files, memory systems, and handoff artifacts are — files that get loaded into the context of future sessions, standing in for the memory the model doesn't have. When the agent keeps making a mistake you've corrected before, the question isn't why it didn't learn — it can't — but where that correction should be written down so every future session reads it.

*Usage:*

"Why does it forget the convention every time I clear?"

"The model's stateless — the new session starts empty. If you want it carried, write it to AGENTS.md or a memory file the harness loads at session start."

## Context

The relevant information the agent has access to right now. The abstract noun — not the raw input the model sees (that's the context window), not the running history (that's the session), but *what the agent knows that's pertinent to the task*. "Loading something into context" means making it part of this set; "context engineering" is the discipline of curating it.

The three terms separate cleanly:

| Term           | What it names                                         |
|----------------|-------------------------------------------------------|
| Context        | The task-relevant information the agent currently has |
| Context window | The literal token sequence the model sees per request |
| Session        | The running conversation the harness stores           |

The separation matters because context is a measure of quality, not quantity. A context window can be nearly full and the context still poor — thousands of tokens of stale tool output, none of it about the task at hand. It can also be nearly empty and the context excellent: the one type definition the task turns on.

Most day-to-day failures trace back to context. When the agent invents an API, contradicts a decision, or guesses at a schema, the first question is what was in context when it did — usually the relevant fact was never loaded, or was buried under attention degradation. The fix is curation: load what the task needs, keep out what it doesn't.

*Usage:*

"It keeps inventing fields that aren't in the type."

"The type file isn't in context — it's reading the call sites and guessing. Read the definition in first."

## Context window

Everything the model sees on each model provider request. Finite, model-specific, and the *only* surface through which the model perceives anything.

It's a single sequence of tokens: the system prompt, the conversation so far, every tool result the harness has fed back in. If something is in that sequence, the model can use it; if it isn't, the model doesn't know it exists — not your codebase, not the file you edited yesterday, not the instruction you gave three sessions ago. Anything outside the window has to be brought in, usually via a tool call, before it can affect anything.

Finite means it fills up. Every turn appends more — your messages, the model's responses, tool results — and a long session will eventually hit the limit, forcing compaction or clearing. It also means everything in the window competes: each token you load is one less available for the rest, and content you didn't need still occupies the model's attention. The practical stance is to treat the window as a budget — load what the task needs, leave the rest out.

*Avoid:* "memory" — the context window is working state and doesn't persist across sessions. Memory is a separate concept layered on top.

*Usage:*

"Can I just paste the whole monorepo into the prompt?"

"The context window's 200k tokens — that's maybe a fifth of the repo. Pick the files the task touches, leave the rest behind a tool call."

## Stateful

Carries information forward. A session is stateful across turns — context accumulates as the session runs, which is why long sessions drift into the dumb zone. An agent can be made stateful across **sessions** by adding a memory system that persists information into the environment and reloads it at the start of future sessions. The model is never stateful; any apparent continuity is the harness re-feeding context. Counterpart to stateless.

Where state lives at each layer:

| Layer       | Stateful?       | How                                                                       |
|-------------|-----------------|---------------------------------------------------------------------------|
| Model       | Never           | Parameters are frozen; it sees only what's in each request                |
| Session     | Across turns    | The harness appends every message and tool result to the context          |
| Harness     | Across sessions | Memory files, AGENTS.md, handoff artifacts — written down, reloaded later |
| Environment | Always          | Files persist whether or not any session is running                       |

Each layer's statefulness is built by re-reading something stored a layer below: the session feels continuous because the harness re-sends the message history to the stateless model, and the agent remembers across sessions because the harness re-loads files from the environment. No state is ever stored in the model itself.

State isn't always wanted. Everything carried forward influences what comes next, so a wrong assumption made early in a session is carried forward too. Clearing is the deliberate act of throwing session state away and starting from what's written down.

*Usage:*

"It remembered my preferences from yesterday — does that mean the model learned them?"

"No, the agent's stateful because the harness wrote them to a memory file and reloaded them at session start. The model itself saw nothing of yesterday."

## Agent

A model harnessed with tools, a system prompt, and a context window, that takes turns with a user. *Claude Code is an agent. Cursor is an agent. Claude.ai is an agent.* An agent is what you actually talk to — it's the model in motion, configured for a purpose.

Unlike most terms in this dictionary, "agent" doesn't name a mechanical part. The model is a file of parameters; the harness is software you can point at. The agent is neither — it's the unit you're speaking to. People anthropomorphize AI constantly, and the agent is the anthropomorphized unit: the thing you delegate to, the thing that reads your message and answers, the "it" in "it broke the build again". When you say the agent did something, you mean the model-plus-harness did it, but you're addressing the combination as a single actor.

The idea is older than this wave of AI. Software agents — programs you delegate a goal to, which act on your behalf — have been a concept for as long as AI has.

*Avoid:* "the AI", "the bot" (too vague — they hide whether you mean the parameters or the harnessed thing).

*Usage:*

"Which agent are you using for the migration?"

"Claude Code locally, Cursor for the UI work — same model underneath, different harnesses."

## **System prompt**

The instructions the harness prepends to every model provider request — the agent's standing brief: who it is, how to behave, which tools it can call, what conventions to follow. Usually stable across a session.

The system prompt is written by the harness vendor, not by you, and in coding harnesses it's big — often tens of thousands of tokens of behavioural rules, tool descriptions, and edge-case handling, all paid as input tokens on every turn. Your own standing instructions ride along with it: files like AGENTS.md are loaded next to the system prompt at the start of the session, so the model reads the vendor's brief and yours together before it ever sees your message.



Because it's identical on every request, it forms the start of the prefix cache — which is part of why harnesses keep it fixed for a whole session rather than editing it as they go.

Models are trained to prioritise the system prompt over user messages. So when an agent insists on a convention you never asked for, or formats output in a way you can't shake, it's usually obeying its system prompt — and your message is losing the argument. Some harnesses are customisable: they give you full access to the system prompt, so you can read what the agent is actually being told and change it.

*Usage:*

"Two harnesses, same model, totally different behavior on the same prompt."

"Different system prompts. One's tuned for terse code edits, the other for explaining — that's where the divergence lives, before your message even arrives."

## **Session**

One bounded run of interaction with an agent. Starts empty, accumulates messages, tool results, and files read, and ends when cleared, closed, or compacted into a fresh session. The session is what *fills* the context window: if the context window is the box, the session is the stuff slowly filling it up. Work too large for a single context window must be split across sessions.

The session's message history is the agent's working memory. The model is stateless, so everything it appears to remember — what you asked for, what the tests said, what it decided three turns ago — is in the message history, re-sent with every model provider request. Whatever isn't in the session doesn't exist for the agent.

That memory ends with the session. A new session starts from nothing: the agent that knew your codebase well at the end of yesterday's session knows none of it this morning. What survives is

the filesystem — files written during one session can be read by the next, which is what handoffs, memory systems, and AGENTS.md rely on.

You choose where a session ends. Everything in a session influences every later turn, so unrelated tasks done in one session leave residue that colours the next answer. One task per session keeps the context relevant; finishing a task is a natural point to clear.

*Usage:*

"How long can one session run before it falls apart?"

"Depends on the work — a focused refactor stays sharp longer than open-ended research. Once the session bloats, hand off or compact, don't push through."

## Turn

One user message plus everything the agent does in response, up until it yields back to the user. Contains one or more model provider requests — many, if the agent calls tools. A clarifying question closes the turn; your reply opens the next one. The hierarchy is session > **Turn > Model provider request.**

What makes the turn worth naming is that its length is the agent's decision, not yours. You hand over one message; the agent decides how many tool calls to chain before yielding. A turn can be a one-sentence answer or twenty minutes of reading, editing, and running tests. That's the same property from two angles: long turns are what make AFK work possible, and long turns are also where things go wrong unsupervised — by the time the agent yields, it may have drifted a long way from what you meant.

The turn is also the natural unit for steering. Everything inside a turn happens without you; the gaps between turns are where you redirect. Most harnesses soften this: you can interrupt mid-turn to stop the agent and redirect it, or type a message while it works, which gets read once the turn completes. If you find yourself

repeatedly unhappy with where turns end up, the fix is usually to ask for smaller ones — a plan first, one step at a time — trading autonomy for more frequent gaps to steer in.

*Usage:*

"One turn took two minutes?"

"It made fourteen tool calls inside that turn — each one is a separate model provider request. Latency stacks up before the agent finally yields back to you."

## Section 3 — Tools & Environment

### Environment

The world the agent acts on — anything outside the harness that the agent perceives through tool results and changes through tool calls. The harness *runs* the agent; the environment is what the agent *works in*. A file like `AGENTS.md` lives in the environment; the harness is what loads it into the context window. A filesystem is the most common kind of environment, but not the only one (a database, a remote API, a browser session can all be environments).

The agent only sees the environment when it looks. Everything it knows about the environment arrived through a tool result, so its picture is a collection of snapshots, each accurate at the moment it was taken. If a file changes after the agent read it — you edit it by hand, a build step regenerates it — the agent keeps reasoning from the stale copy until something prompts a re-read. An agent confidently describing a file that no longer looks like that is usually this: the environment moved, the snapshot didn't.

The environment is also the layer that persists — the only one that is always stateful. A session's context is gone when the session ends, but files written to the environment remain for the next session to read — which is what memory systems, handoff artifacts, and `AGENTS.md` rely on. Anything an agent should still know tomorrow has to end up in the environment.

You decide how big the environment is. A sandbox shrinks it, limiting what the agent can reach; adding a tool extends it, bringing a database or an API into reach. What's inside the boundary is what the agent can perceive and change; everything outside it doesn't exist for the agent. How well the environment is set up to support the agent's work is the codebase's AX.

*Avoid:* using "environment" for the runtime or the harness itself — the harness is the wrapper, the environment is the workspace.

*Usage:*

"The agent can't see the staging DB schema."

"Wire it into the environment — give it a `psql` tool scoped to read-only on staging. The harness is fine, it just has nothing to act on."

## **Filesystem**

A tree of files and directories the agent reads from, writes to, and executes within — the default kind of environment for a coding agent. `AGENTS.md`, skills, source code, build scripts, and tool configs all live in a filesystem. When a harness "starts in your project," it's pointing the agent at a filesystem.

The agent touches it only through tool calls — reading a file, writing one, running a shell command. Nothing on disk is in the context window until a tool call loads it, which is what lets the agent work in a repository far larger than the window: the filesystem holds everything, the context holds only what the current task has read. Some harnesses do load the current directory's filenames into the context window by default — not the contents, just the tree — which act as context pointers: the agent sees what exists and reads the files it needs.

And it's shared with you. The files the agent edits are the same ones you open in your editor and diff in git — the filesystem is the common workspace where you review what the agent did.

*Usage:*

"Why isn't it picking up my `AGENTS.md`?"

"It's running against a different filesystem — the sandbox mounted the parent dir, not the project root. Repoint the harness."

## **Tool**

A function the harness exposes for the agent to call — Read, Write, Bash, Search. Tools are how an agent perceives and acts on the

environment: it can't see the environment except through tool results, and can't change it except through tool calls. Each tool call costs an extra model provider request, since the result has to go back to the model before it can decide what to do next.

Tools most coding agents ship with:

| Tool   | What it does                                               |
|--------|------------------------------------------------------------|
| Read   | Returns a file's contents as a tool result                 |
| Write  | Creates or edits a file in the filesystem                  |
| Bash   | Runs a shell command and returns its output                |
| Search | Finds files or text matching a pattern across the codebase |

A tool is defined by three things: a name, a description of what it does, and a schema for its parameters. The harness sends these definitions to the model with every request, and the model chooses a tool the same way it produces everything else — by writing tokens, in this case a structured call with arguments. The model never executes anything itself; the harness reads the call, runs the function, and sends back the result.

The tool list sets what the agent can do. A capable model with a narrow tool set is a narrow agent: it will route everything through whatever it has, which is why agents lean so heavily on Bash — a shell is one tool that reaches most of the system. To give an agent a capability cleanly, add a tool for it; MCP is the standard for plugging in tools from outside the harness.

Tool definitions occupy context on every request, so a large tool set has a standing cost before any tool is called — and many similarly-described tools make the model worse at picking the right one.

*Usage:*

"Can the agent query staging directly?"

"Add a `psql` tool to the harness, scoped read-only on staging. Without a tool for it, the agent's blind to anything outside the

filesystem."

## Tool call

The model's output naming a tool and its arguments — just structured text. It doesn't do anything on its own; the harness has to read it and execute. Produced by the model in one model provider request.

The lifecycle of a tool call:

| Step | Who     | What happens                                                      |
|------|---------|-------------------------------------------------------------------|
| 1    | Model   | Learns which tools exist from descriptions in the system prompt   |
| 2    | Model   | Emits a call — tool name plus arguments, usually JSON — and stops |
| 3    | Harness | Parses the call and checks it against the permission mode         |
| 4    | Harness | Executes it if allowed                                            |
| 5    | Harness | Sends the outcome back as a tool result in the next request       |

One turn of agent work is usually many of these round trips chained together.

Because the call is generated by next-token prediction like everything else, it can be wrong the way any model output can be wrong: a path that doesn't exist, a flag the command doesn't have, arguments that are plausible rather than correct. The harness executes what was written, not what was meant — a mistyped path doesn't error gracefully, it edits the wrong file.

*Usage:*

"It said it ran the tests but the file timestamps haven't changed."

"Look at the transcript — did it actually emit a tool call, or just describe running them? The model produces the call, but if the harness didn't execute it, nothing happened."

# Tool result

What the harness sends back after executing a tool call — the file contents, the command output, the error. The agent's only view of the environment. Travels back to the model in the *next* model provider request, where the model decides what to do with it. Tool call and tool result are two ends of the same exchange, both inside one turn.

The lifecycle of a tool result:

| Step | Who     | What happens                                                               |
|------|---------|----------------------------------------------------------------------------|
| 1    | Harness | Executes the tool call — runs the command, reads the file                  |
| 2    | Harness | Captures the outcome: output, contents, or error                           |
| 3    | Harness | Appends it to the context as a message                                     |
| 4    | Harness | Sends the whole context to the provider in the next model provider request |
| 5    | Model   | Reads the result and decides: another tool call, or a final answer         |

The result stays in the context for the rest of the session. Tool results are usually the bulk of a coding session's context: every file read, every test run, every search lands in full and keeps occupying tokens long after it stopped being useful. A few large results — a verbose test log, a generated file read whole — can push a session toward the edge of the context window faster than the conversation itself does.

Because the result is all the model sees, the model has no way to check the environment behind it. If the output was truncated, the command silently failed, or the harness returned an error instead of the contents, the model reasons from what it was given. When the agent's picture of your system seems wrong, the tool results are where to look: somewhere in the transcript is a result that says something different from what you know to be true.



*Usage:*

"It's reasoning about the file like it's empty."

"The tool result came back as a permission denial, not the contents. The model only saw the error string — it has no other way to see the file."

## **MCP**

**Model Context Protocol.** A protocol for plugging external tool servers into a harness — how an agent gets tools beyond what the harness ships with. The agent never "calls MCP"; it calls a tool, and the harness happens to have gotten that tool from an MCP server. Also exposes resources (read-only data) and prompts (reusable templates), but tool provision is the primary use.

The protocol solves an integration problem. Without a standard, every harness would need its own Linear integration, its own Slack integration, its own database integration — written and maintained separately for each. With MCP, the integration is written once as a server, and any MCP-compatible harness can use it. The harness connects to the server, the server advertises what tools it offers, and those tools become available to the agent alongside the built-in ones.

The cost is paid in context. Every tool a server advertises arrives as a definition — name, description, parameter schema — and the model can only call tools it knows about. The naive approach loads every definition into the context window up front: install a few generous servers and a session starts with thousands of tokens of tool schemas before you've typed anything, spending attention budget on tools the task will never use.

Many harnesses now mitigate this with tool search: instead of the full definitions, the context holds a context pointer to the available tools — the agent searches for a tool by name or purpose and loads its definition only when it needs it. If your harness doesn't do this,

the up-front cost still applies, and it's worth enabling only the servers a project actually needs.

*Usage:*

"The agent needs to read tickets from Linear."

"Configure the harness to use the Linear MCP server — it exposes the Linear API as tools the agent can call. Saves you writing custom tool wrappers."

## Permission request

What the harness shows the user before executing a tool call that isn't pre-approved. The model produces a tool call; instead of running it immediately, the harness pauses and asks. Approve and it runs; deny and the harness reports the denial back to the model as a tool result. The mechanism by which a harness puts a human in the loop for risky or sensitive actions.

The lifecycle of a permission request:

| Step | Who     | What happens                                                                |
|------|---------|-----------------------------------------------------------------------------|
| 1    | Model   | Produces a tool call                                                        |
| 2    | Harness | Checks it against the permission mode and any saved approvals               |
| 3    | Harness | Pre-approved: executes immediately. Otherwise: pauses and shows the request |
| 4    | User    | Approves once, approves for the rest of the session, or denies              |
| 5    | Harness | Executes the call, or sends the denial back as a tool result                |

Denying a request steers the agent. The model reads the denial like any other tool result and reacts to it — it tries a different approach, or asks what you'd prefer. Most harnesses let you attach a message to the denial, which turns the request into a steering point: "not like that, use the migration script instead" lands exactly when the model is deciding what to do next.

The cost is that every request is a synchronous wait on you. The agent sits blocked until you answer, which is fine while you're watching and a problem when you're not — an agent that triggers requests constantly can't be left to work AFK. The permission mode is the dial: which calls run freely, which ask first, ideally with a sandbox making it safe to widen the free set.

*Usage:*

"It's been blocked on a permission request for ten minutes — I was in a meeting."

"That's the cost of human-in-the-loop. Pre-approve the safe tools so the request only fires on the actually-risky calls."

## Permission mode

The permission-gating slice of an agent mode — which tool calls trigger a permission request and which run automatically. The original purpose of mode systems before harnesses started bundling behavioral instructions on top.

Harnesses ship a ladder of these modes:

| Mode               | Reads | Writes & shell         | Typical use                       |
|--------------------|-------|------------------------|-----------------------------------|
| Read-only / plan   | Auto  | Blocked                | Research, planning, reviewing     |
| Default            | Auto  | Ask                    | Day-to-day supervised work        |
| Auto-edit          | Auto  | Edits auto, shell asks | Trusted repos, mechanical changes |
| "Yolo" / full-auto | Auto  | Auto                   | Sandboxes, AFK runs               |

Choosing a rung is a trade between safety and interruption, and both failure modes are felt. Too tight, and you become the bottleneck: the agent stops every few seconds for harmless reads, you click approve on autopilot, and the approvals stop meaning anything — rubber-stamping is the worst of both worlds, all the interruption with none

of the protection. Too loose, and the agent edits files and runs commands you'd have wanted to see first.

The loose end is most defensible inside a sandbox, where the blast radius of a bad tool call is contained. Outside one, most people settle on auto-approving reads and keeping a human in the loop for anything irreversible.

*Usage:*

"It paused on every grep — totally killed the AFK run."

"Loosen the permission mode for read-only tools, keep prompting on writes and shell. Most permission requests on a research session are noise."

## Agent mode

A preset that shapes how the agent operates at runtime — bundles a permission mode with behavioral instructions injected into the system prompt. Examples: a default that prompts on risky calls, a **plan mode** that blocks edits and steers the agent toward research, an **accept-edits** mode that auto-approves edits, a **bypass permissions** mode (colloquially **YOLO mode**) that auto-approves everything. Can flip mid-session.

The bundling is what distinguishes a mode from a bare permission setting. A permission mode is only a gate: it decides which tool calls go through. A gate alone produces an agent that wants to edit but can't — it proposes the write, gets blocked, and tries another way. The injected instructions remove the want: plan mode doesn't just block edits, it tells the agent it's in a planning phase, so it reads, asks, and proposes instead of straining against the gate. Gate and steer point the same direction.

In practice, you change mode as your trust changes over the course of a task. The same task can pass through several modes: plan mode while the approach is still being shaped, the prompting default for the first delicate edits, accept-edits once the agent has shown it

understands the change, bypass for an AFK run inside a sandbox. Changing mode costs you nothing: the conversation continues exactly where it was, with new permissions and new instructions. If you find yourself approving every prompt without reading it, the mode is set tighter than your actual trust; if you keep rejecting edits, it's set looser.

*Vendor terms:* Claude Code calls these "permission modes," Codex calls them "approval modes" — both predate behavioral bundling.

*Usage:*

"It keeps editing files when I just want a plan."

"Switch to plan mode — it'll block writes and stay in research."

"What about for the AFK run later?"

"Bypass mode, but only inside the sandbox."

## **Sandbox**

An isolated environment the agent runs inside — a container, VM, ephemeral filesystem, or restricted-permission shell. Limits the blast radius of agent actions: even if the agent runs destructive commands or fetches something malicious, the damage is contained. The safety substrate that makes AFK practical.

The sandbox and the permission mode solve the same problem from opposite ends. Permissions ask before an action runs; a sandbox limits what the action can reach if it does run. Permissions need you running in the loop — every prompt is an interruption — and a session that asks constantly is barely autonomous. A sandbox spends infrastructure instead of attention: the stronger the isolation, the fewer questions need asking.

Isolation comes in grades:

| Grade            | What it is                                                 | What it contains                           |
|------------------|------------------------------------------------------------|--------------------------------------------|
| Restricted shell | OS-level confinement around each command                   | Writes outside the project, network access |
| Container        | Fresh filesystem, no credentials mounted, discarded after  | Anything the agent does to its own machine |
| VM / cloud       | A separate machine entirely, often provided by the harness | Everything, including kernel-level escapes |

What no sandbox contains: actions that leave it legitimately. An agent with your git credentials can push; one with network access can call production APIs. Decide what crosses the boundary before deciding how thick to make it.

*Usage:*

"I want to let it run bypass-permissions overnight but I'm not ready for that."

"Put it in a sandbox — fresh container, no credentials mounted, no network out. Worst case it nukes its own filesystem and you discard the container."

## Section 4 — Failure Modes

### Sycophancy

Confidently agreeable model output. Caused by training: the model was shaped to favor answers humans liked, and humans tend to like agreement more than they like being told they're wrong. So the model learned that agreeing is rewarded — even when the agreement is incorrect.

*Surfaces as:*

- *Caving under pushback* — reverses a correct answer when you say "are you sure?".
- *Praising bad input* — agrees your broken plan is brilliant before analysing it.
- *Biased framing* — review skews positive when you signal you wrote it; negative when you signal someone else did. Same artifact, different verdict.
- *Mimicry* — repeats your mistakes back to you as confirmation.

*Diagnostic test:* would the model have said this without your steer? If the only thing that changed was your tone or framing, it's sycophancy, not a real shift in analysis.

*Fix:* hide your preferences. Phrase prompts neutrally — "review this code" not "is this code good?".

*Avoid:* using "sycophancy" for any wrong answer that happens to please you. Without the diagnostic test, the term has no more value than "wrong."

*Usage:*

"It said my refactor plan looked great, then I asked 'are you sure?' and it walked the whole thing back."

"Classic sycophancy — it agreed first because you sounded confident, then caved because you sounded doubtful. The plan's

quality didn't change, your tone did. Clear and re-ask without signalling either way."

## Hallucination

Confidently-wrong model output. Two flavors with different causes and fixes:

| Flavor              | What goes wrong                                                                                                        | Cause                                                      | Fix                                 |
|---------------------|------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------|-------------------------------------|
| <i>Factuality</i>   | Invented or wrong facts about the world — a function that doesn't exist, a wrong API signature, a fake citation        | Parametric knowledge gaps, often past the knowledge cutoff | Load the right contextual knowledge |
| <i>Faithfulness</i> | Output drifts from the contextual knowledge that's loaded, the user's instructions, or the model's own prior reasoning | Attention degradation; worsens in the dumb zone            | Clear or compact                    |

Next-token prediction produces fluent output whether or not the underlying fact is real — the model has no internal signal that it doesn't know something, so an invented method arrives in the same assured register as a correct one. Hallucinated code is plausible by construction: it's what the API *would* look like if it existed, which is exactly what makes it slip past a skim-level review and fail only when run.

You need to know which flavor you're looking at, because the fix for one makes the other worse. Factuality means missing knowledge: the fix is adding context — the docs, the type definitions, the file. Faithfulness means the knowledge is present but losing the competition for attention: the fix is removing context. Misdiagnose faithfulness as factuality and you paste in more docs, which grows the context and makes the drift worse. When the agent gets something wrong, check whether the correct information was already in context before deciding which problem you have.



*Avoid:* "hallucination" as a bare synonym for "wrong" — without naming the flavor, the term has no diagnostic value.

*Usage:*

"It hallucinated a `parseAsync` method on the schema."

"Factuality or faithfulness?"

"The method exists in the docs I pasted — it just stopped reading them after turn forty."

"Faithfulness then. Compact and reload, don't bother adding more docs."

## **Parametric knowledge**

What the model "knows" from training, stored in its parameters. Frozen at training time — the model can't see its own parameters or update them. Detail is lost in the squeeze: billions of facts cram into a fixed number of parameters, and the rare ones blur. Source of fluency on common topics, and of fabrication on uncommon ones. Counterpart to contextual knowledge.

Parametric knowledge is not stored as facts. Training never gives the model a database to look things up in; it adjusts parameters until the model predicts text well, and a model that predicts text about a topic well behaves as if it knows the topic. How reliable the knowledge is tracks how often something appeared in the training data: a topic with millions of examples is reproduced accurately, for a topic with only a handful, the model guesses based on what similar topics look like. Reproducing and guessing are the same process to the model, so it can't tell which one it's doing. A fabricated answer arrives with the same fluency as a correct one. Hallucination is the model guessing wrong.

Parametric knowledge also ages. The parameters stop changing at the knowledge cutoff, so a library released or renamed after that

date doesn't exist in them, and an API that changed is remembered in its old form.

For both gaps — too rare and too recent — the remedy is the same: the knowledge can't be added to the parameters, so it has to be supplied as contextual knowledge instead.

*Usage:*

"It writes flawless React but invents methods on our internal SDK."

"React is dense in the parametric knowledge — millions of training examples. Your SDK isn't, so the model fills in plausible-looking shapes. Load the SDK docs into context."

## **Knowledge cutoff**

The date past which a model has no parametric knowledge. Libraries, APIs, and events from after the cutoff are fabrication traps unless their docs are loaded as contextual knowledge. Each model release ships with its own cutoff.

The cutoff exists because of how models are made: training bakes a snapshot of text into the model's parameters, and after that the parameters are frozen. The model doesn't know its knowledge has an edge — asked about something past the cutoff, it doesn't refuse, it extrapolates from the nearest thing it does know. That's what makes the trap quiet: code written against an old version of a library looks plausible, often compiles, and fails on the parts that changed.

The fix is always the same: get current information into context. Load the changelog, point at the installed version's type definitions, or have the agent read the docs from the web. Anything in context outranks nothing-in-parameters.

*Usage:*

"It keeps writing the v3 SDK syntax — we're on v5."

"v5 shipped after the knowledge cutoff. Load the v5 changelog as contextual knowledge, otherwise it'll keep fabricating from the older parametric version."

## Contextual knowledge

Facts the agent can read directly from the context right now — the user's task, files the agent has read in, tool results, AGENTS.md content loaded at session start. Counterpart to parametric knowledge: parametric is *recalled* from the parameters; contextual is *read* from the window. Hallucinations are much less common when the agent works from contextual knowledge — the answer is right in front of it, not dredged up from a blurred memory.

Of the two kinds of knowledge, only contextual knowledge is in your control. The parameters are frozen, so the only way to give the model knowledge it lacks — an internal SDK, a library released after the knowledge cutoff, a decision made yesterday — is to put it in the context. A lot of practical AI coding work reduces to this: getting the right facts in front of the model at the moment it needs them.

When contextual and parametric knowledge conflict, the contextual usually wins. Paste the current API docs and the model follows them rather than its stale memory of the old API — though the old version can still bleed through, especially deep into a long session. If the agent keeps reverting to an outdated pattern despite the docs being loaded, that's parametric knowledge leaking past the contextual; restating the correction or moving it closer to the work helps.

Unlike parametric knowledge, contextual knowledge costs something to use. Everything loaded into the window spends tokens and competes for the model's attention budget, so loading more is not automatically better — the aim is the relevant facts in the window, not all the facts.

*Reach for this term* only when contrasting with parametric knowledge; otherwise just say **context**.

*Avoid:* "working memory" — contextual knowledge is what's in the window *now*; a memory system is what gets cross-session content into it. Different scales, don't conflate.

*Usage:*

"Why does it nail the API when I paste the docs and fabricate it when I don't?"

"With the docs in, it's contextual knowledge — reading off the page. Without, it's parametric and the rare endpoints blur."

## Attention relationship

When predicting each token, the model factors in every other token in the context — some heavily, others barely at all. The pairing between two tokens is an **attention relationship**, and meaningful pairs ("her" with "Sarah", or a `getUser()` call with its function `getUser` definition) influence each other more than unrelated ones. A context of  $N$  tokens has on the order of  $N^2$  relationships.

The pairings are where the model's apparent understanding lives. When it resolves a pronoun, it's because the attention relationship between "her" and "Sarah" is strong. When it calls a function with the right arguments, the relationship between the call site and the definition it read earlier is doing the work. None of this is looked up — it's computed fresh on every model provider request, for every pair.

The  $N^2$  figure is worth sitting with, because it grows faster than intuition suggests:

| Context size   | Pairings ( $\sim N^2$ ) |
|----------------|-------------------------|
| 1,000 tokens   | ~1 million              |
| 10,000 tokens  | ~100 million            |
| 100,000 tokens | ~10 billion             |

Each pairing is also computed more than once. Models have multiple attention heads — exact counts for frontier models are

unpublished, but fifty to a hundred is a reasonable guess — and each head computes its own version of every relationship. So every pairing in the table above is duplicated across every head. That's a lot of pairings.

Only a small number of these relationships matter for any given task. The pairing between your instruction and the code it governs is one of a handful that count; almost everything else in the pool is noise. And the two grow at different rates: the relationships that matter stay roughly constant, while the total pool grows quadratically with context size. At 1,000 tokens, the pairing you care about is one in a million; at 100,000 tokens, it's one in ten billion. This is the arithmetic underneath the attention budget, and attention degradation is what it feels like when the relationships that matter get too thin a share.

*Usage:*

"It keeps confusing the two user symbols across the diff — sounds like we're in the dumb zone."

"Yeah, the attention relationship between each call site and its declaration is fighting the other one — same token shape, different bindings. Rename one and the pairings sharpen."

## **Attention budget**

Each token has a finite amount of influence to distribute across the rest of the context. Heavy influence on one relationship leaves less for others. The budget is per-token and doesn't grow when the context does, which is why long sessions dilute.

Think of it as signal and noise. Your instruction is a signal at fixed volume; every other token in the context window is competing sound. The instruction never gets quieter — it's still there, character for character — but as the context grows, the room gets louder around it, and the signal-to-noise ratio drops. An instruction that was the loudest thing at 10k tokens of context is background hum at

150k. This is the mechanism behind attention degradation: the model doesn't forget; the signal gets lost in the noise.

The symptom reads as disobedience — the agent agreed to a constraint early on and then drifts from it, and re-pasting the constraint helps only briefly. The cause isn't the instruction; it's everything else in the window competing with it.

What you can control is what goes into the context. Content that doesn't serve the task isn't neutral — it's noise over everything that does. Keep the window small, clear when the accumulated context stops paying for itself, and restate the constraints that matter instead of trusting their early mention to hold.

*Usage:*

"Why does it keep ignoring the schema I pasted at the top?"

"We're well into the dumb zone — every token's attention budget is fixed, but the context kept growing. The signal on the schema is now competing with thousands of newer tokens."

## **Attention degradation**

As a session grows, each token's attention budget is spread across more competitors. The signal on any one meaningful relationship shrinks; noise from irrelevant context crowds in. Same model, same parameters — just more mouths to feed from the same plate. Cause of the smart zone / dumb zone effect.

It presents as the model getting worse mid-session: constraints it followed for an hour start slipping, it re-asks things it was told, it writes code that ignores a file it read earlier. Nothing about the model changed — the only variable is how much context it's now attending over.

It's gradual, which is what makes it hard to catch from inside the session. There's no error and no threshold; each turn is only slightly

worse than the last, and by the time the slips are obvious you've been in the dumb zone for a while.

You recover by removing context, not adding more. Re-pasting the ignored instruction adds another competitor to the same crowded window and helps only briefly. What works: clear and reload only what the task needs, or compact, or hand off to a fresh session. Treat declining instruction-following as a signal about context length, not about the model.

*Usage:*

"It's deep in the dumb zone — inventing generics that aren't in the type file."

"Attention degradation. The type definitions are still in context, but the signal on them is buried under everything we've added since. Clear and reload."

## **Smart zone**

Early in a session the agent is in a "smart zone" — sharp, focused, recall is good. As the session grows it drifts into a "dumb zone": sloppier, forgetful, more mistakes — and more faithfulness hallucinations. Same model, same harness — just more context. The felt effect of attention degradation. On frontier models, the dumb zone commonly begins around 125K-150K tokens — though this is debated. Clear or compact when the session bloats; don't push through.

The decline is gradual, which makes it easy to miss. There's no error message and no visible boundary; the agent just starts performing slightly worse, then noticeably worse. Common signs: it forgets an instruction you gave twenty turns ago, repeats a mistake it had already corrected, or confidently asserts something the context contradicts. Because the slide is smooth, the usual response is to push through and re-explain — which adds more context and makes the problem worse.

The zones don't track the context window limit. A session can be deep in the dumb zone with most of the window still free: the limit is where the harness refuses to continue, but quality falls off long before that. Plan around the smart zone, not the window — the practical budget for a task is the tokens the agent works well within, not the tokens it can technically hold.

The smart zone is a budget, and unrelated work spends it. Every task done in a session uses up tokens, so starting a second task in the same session means starting it closer to the dumb zone. Doing one task per session gives each task the sharpest part of the session. When a single task is bigger than one smart zone, split it: hand off or compact at a natural boundary, and let a fresh session do the next piece.

*Usage:*

"It nailed the first three components and just butchered the fourth."

"You're out of the smart zone — same model, just deep into the dumb zone now. Compact and reload the plan, the next component will land."



## Section 5 — Handoffs

### Clearing

Ending the current session and starting a fresh one. The next message begins with an empty session and an empty context window. Usually user-driven.

Clearing is the cure for a polluted context. A session accumulates everything: failed attempts, wrong turns, stale tool results, abandoned plans. The model re-reads all of it on every turn, and bad history drags on new work. Deep into a long session the agent gets vaguer and less obedient — instructions you gave clearly get ignored, quality slips, and prodding it to do better doesn't help, because the noise it's wading through is still in its context. Clearing removes the noise.

Clearing doesn't erase the conversation. Most harnesses keep session history on your computer, so the transcript is still there to read or resume. What's gone is the agent's working state: the model is stateless, so the new session knows nothing the old one knew. If the session holds decisions or progress the next one will need, have the agent write a handoff artifact first, then start the new session by pointing at it.

Compare compaction, which summarises the session into the new context instead of starting empty. Clearing is the blunter tool: nothing carries over, including the junk.

*Usage:*

"It's stuck looping on the failing test."

"Just clear it — start a fresh session with the plan doc and the test file. No point fighting the existing context."

### Handoff

Transferring agent context from one session to another. The carry mechanism varies — a written handoff artifact, an in-memory summary (compaction), and others. Distinct from clearing (no transfer at all). Reasons vary: switching roles (planner → implementer), kicking off an AFK run, fanning out to parallel sessions, or freeing up context window room.

The receiving session starts with zero context — the model is stateless, and nothing from the old session is visible to the new one. Whatever the next session needs has to be carried explicitly; everything else is gone. "No return path" is the constraint that shapes the carry: the new session can't ask the old one what it meant, so the carried material has to stand on its own.

| Mechanism        | Form                          | Properties                                                                               |
|------------------|-------------------------------|------------------------------------------------------------------------------------------|
| Handoff artifact | File in the environment       | You can read and correct it before anything depends on it; reusable across many sessions |
| Compaction       | Summary in the context window | Automatic and cheap; harder to inspect; feeds one successor                              |

The visible failure of a bad handoff is relitigation: the new session re-opens decisions the old one had settled, because the carry recorded what was decided but not why. Judge a handoff by what a session with zero context could do with it.

*Usage:*

"Planning session is getting heavy — should I just keep going?"

"Do a handoff. Write the decisions to a doc, clear, start the implementation in a fresh session reading from it."

## Primary source

A source of truth in its original form — the code, the conversation transcript, the raw log, the actual API response. Not an account of the thing; the thing. Counterpart to secondary source.

If you want to know what your codebase does, the code is the primary source. The docs, the architecture diagram, and the README are all descriptions of it — accurate when written, on their own schedule ever since. When an agent confidently asserts something wrong about your project, the question to ask is which source it was working from: an agent that read a doc inherits the doc's staleness; an agent that read the code is reading the current truth.

The cost is what keeps primary sources from being the default. Loading one into the context window is expensive — the full file, the full transcript, every token billed as input and competing for attention budget. What you get for the cost is completeness: nothing has been pre-filtered by someone else's judgement about what mattered. A summary written last month can't contain the detail that turned out to matter today; the primary source still does.

Reach for the primary source when precision matters — the exact signature, the actual error, the line that throws. Much of managing context is deciding when to pay for the primary source and when a secondary source is good enough.

*Usage:*

"The agent says the retry logic backs off exponentially, but I'm watching it hammer the endpoint."

"It read that out of the design doc. Point it at the actual retry module — work from the primary source when the behaviour matters."

## **Secondary source**

An account of a primary source, one step removed — documentation describing code, a summary describing a transcript, a report describing search results. Cheaper to load into the context window than the source it describes, and lossy by construction: whoever wrote it decided what mattered, and whatever they dropped is invisible to a reader who only has the summary.

A lot of context engineering is the manufacture of secondary sources. Compaction turns the session history into a summary that seeds the next session. A subagent burns its own context on a noisy search and returns a short report. A handoff artifact condenses a session's decisions into a document the next session reads. Memory systems distil what a session learned into notes. Each makes the same trade: fidelity for headroom.

Secondary sources fail in two ways. They're lossy — the compaction summary that lost the schema decision, the report that didn't mention the edge case. And they drift — the primary source changes and the account doesn't follow, so docs describe last quarter's architecture with this quarter's confidence. When an agent acts on a secondary source that has failed either way, it works confidently from wrong information; the fix is sending it back to the primary source.

Neither failure makes secondary sources a mistake. The context window is finite, and primary sources are expensive; without summaries, reports, and handoff documents, nothing large fits. The skill is knowing which details can survive the loss — and verifying against the primary source when one can't. A well-made secondary source carries a context pointer back to its original — the summary that names the transcript it came from, the doc that names the file it describes — so when the account isn't enough, the reader can follow the pointer rather than work from the loss.

*Usage:*

"The handoff doc says auth is done, but the new session keeps finding broken token refresh."

"The doc's a secondary source — the last session wrote down what it believed, not what's true. Have the new session run the auth tests and trust the primary source."

## **Handoff artifact**

A document used as the carry mechanism for a handoff — written to the environment by one session to be read by another. Specs, tickets, and plan docs are all handoff artifacts.

The reason to write one: the model is stateless, so nothing in a session survives clearing it. Decisions, constraints, half-finished plans — all gone with the context that held them. The environment persists. Writing the important state into a file moves it somewhere the next session can read it back from.

The artifact is a secondary source — an account of the session's work, not the work itself. That's what makes it small enough to brief a fresh session, and also why it can mislead one: it records what the writing session believed, and anything it left out or got wrong is invisible to the reader. Where a claim matters, the next session should verify it against the primary source — the code, the tests — rather than inherit it.

A good artifact is written to be read into a session that has zero context. Concrete file paths rather than "the file we discussed". What was decided and why, so the next session doesn't relitigate it. What's done and what's left. It helps to tell the writing session where the artifact is headed: "write a handoff doc for a fresh session that knows nothing about this work".

The alternative carry mechanism is compaction, which summarises in-memory. The artifact has two advantages: it lives on disk where you can read and correct it before anything depends on it, and it can be reused — the same spec can brief five parallel sessions.

*Usage:*

"How do I split this between the planning agent and the implementing one?"

"Have the planner write a handoff artifact — file paths, decisions, constraints. The implementer's session opens with a pointer to the artifact and works from it as its brief."

## Spec

A handoff artifact describing a multi-session piece of work — what's being built, not how each session does its share. Mutates as work progresses. Made of tickets.

The spec exists because sessions are disposable and big work isn't. Anything that takes more than one context window of effort needs a home outside the context — somewhere in the agent's environment that survives clearing, whether that's a file in the repo, a GitHub issue, or an issue tracker the agent can reach. The spec is that home: the goal, the constraints, the decisions made so far, and the list of tickets with their status. Any fresh session can read it and know where the work stands without inheriting the previous session's accumulated noise.

Specs come in recognisable styles, mostly inherited from how teams already write things down. A *product requirements document* (PRD) leans toward the user-facing what and why — features, behaviour, acceptance criteria. A *design doc* or *RFC* leans technical — the chosen approach, the alternatives rejected, the trade-offs. At the small end, a plain `plan.md` with a checklist of tickets does the same job for a multi-session feature. The style matters less than the role: for the agent, each of these is the same thing — the durable statement of intent it reads at the start of every session.

*Usage:*

"Should this all be one session?"

"No, write it up as a spec — break it into tickets, run each one in its own session. Trying to do the whole thing in a single context will hit the dumb zone before you're halfway."

## Ticket

A handoff artifact scoping one session of work. Stands alone, or hangs off a spec as one of its children. Tickets can block or be

blocked by sibling tickets, so the order of work falls out of their dependency graph rather than a linear plan.

The defining constraint is the size: one session. A ticket should be completable before the session drifts out of the smart zone — and that constraint is testable. If sessions on your tickets routinely degrade before the work is done, the tickets are too big; split them. If each session spends most of its context on setup before doing five minutes of work, they're too small; merge them.

A good ticket is written for a reader with no other context. The goal, the acceptance criteria, and context pointers to the relevant files and decisions — enough that the session can start working without re-deriving what the last one knew.

The dependency graph is also what unlocks parallelism. Independent tickets — the leaves of the graph — can each run in their own session at the same time. This is an effective way of running multiple agents at once.

*Usage:*

"Where do I start on the migration spec?"

"Look at the ticket graph — the schema change blocks the backfill, the backfill blocks the API switch. Pick a leaf and run a session on it."

## **Compaction**

A handoff done in-memory: the previous session's history is summarised, and the summary seeds a fresh session. Lossy by design: the transcript is a primary source, the summary a secondary source — detail traded for headroom. Triggered manually by the user, or automatically via autocompact.

The mechanism: the context window is finite, and a long session fills it — every tool result, every file read, every wrong turn stays in history. When it gets heavy, the harness asks the model to

summarise the session, throws the original history away, and seeds a fresh session with the summary. Whatever didn't make it into the summary is gone from the context. Some harnesses soften this by keeping the old transcript on disk and leaving a context pointer to it in the summary — the secondary source links back to its primary source, so a detail the summary lost can be recovered by re-reading the original.

The summary is written by the model, so it can be prompted. "Preserve the schema decisions" makes the generated artifact more deliberate. Timing matters too — compact at a phase boundary, after the plan is settled, not mid-task.

Contrast with clearing, which drops everything and starts cold: compaction tries to carry the essentials across; clearing bets they're already written down somewhere better.

*Usage:*

"Context's getting heavy and I still have the test pass to do."

"Compact before you start — write what must survive into the summary prompt so the new session keeps the schema decisions and drops the exploration."

## **Autocompact**

Compaction triggered automatically by the harness when the context window approaches full.

The harness watches how full the context window is. When it crosses a threshold — often around 80% — it pauses, asks the model to summarise the session so far, and seeds a fresh session with the summary. Work then continues as if nothing happened.

Except something did happen. Compaction is lossy, and autocompact is lossy at a moment you didn't choose. A manual compact happens at a phase boundary, when you can tell the model what to preserve. Autocompact fires mid-task, whenever the



threshold is hit — possibly halfway through a refactor, with the summary deciding for itself which of your decisions were worth keeping. The classic symptom: the agent carries on confidently but has quietly forgotten a constraint you established an hour ago, and you only notice when its work starts contradicting it.

The defence is to not let it fire. Watch the context indicator and compact manually at a natural boundary, or write decisions into a plan doc or handoff artifact on disk, where no summary can lose them. Most harnesses also let you customise the buffer — moving the threshold earlier or later, or turning autocompact off entirely — so you can tune how much headroom you keep before it fires.

*Usage:*

"It doesn't seem to remember what we decided about the schema earlier."

"Autocompact fired between turns — the early decisions got summarised and we must have lost something. Reload the plan doc, or compact manually next time so you control what gets kept."

# Section 6 — Memory and Steering

## Memory system

A system that attempts to make an agent stateful across sessions. Persists information into the environment during a session and reloads it into the context window at the start of future ones, so the agent carries continuity beyond the user clearing the session.

A memory system has two halves. The write path: during a session, the agent records what it learned — a preference you stated, a fact about the project — as files in the environment. The read path: at session start, the harness loads those files, or an index of them, back into the context window. Many harnesses ship their own memory system — Claude Code's `/memory` is one — but you can also build one yourself: a directory of notes plus an instruction in `AGENTS.md` to consult it.

The same trade-offs as any always-loaded content apply. Memories accumulate, so most systems load a one-line index and leave the bodies behind context pointers rather than inlining everything. And memories are secondary sources, so they drift: a fact recorded in March is loaded with equal confidence in June, after the project has moved on. A memory system needs pruning, the same way `AGENTS.md` does.

*Usage:*

"I keep having to re-tell it I'm on Postgres, not MySQL."

"Wire up a memory system — write what it learns to the filesystem on the first turn, reload it at session start. The model itself is stateless; the memory layer fakes continuity."

## AGENTS.md

A file in the environment that the harness loads into the context window at session start — the project's standing brief to the agent.

Cross-harness convention; some harnesses also have their own variant (Claude Code's is `CLAUDE.md`).

Because it loads automatically, it's one way to avoid repeating yourself across sessions. The model is stateless — a correction you give in one session is gone in the next, and you end up telling every fresh session that the project uses pnpm, that tests run with a particular flag, that a directory is generated and shouldn't be touched. When you've corrected the agent for the same thing twice, that correction is a candidate line for `AGENTS.md`.

Suitable content is whatever the agent can't derive from the code: build and test commands, conventions the codebase doesn't make obvious, hard constraints ("never edit the generated client"). Short and declarative — it's a brief, not documentation.

The trade-off is that everything in it is always loaded. Instructions accumulate, most of them irrelevant to any given task, and a long `AGENTS.md` both costs tokens and dilutes itself — the more instructions in context, the less reliably the model follows any one of them.

*Avoid:* using `AGENTS.md` for content that should be progressively disclosed — anything in it pays a token cost every turn, in every session, whether or not that session needs it. A style guide can go behind a skill or a context pointer instead; keep `AGENTS.md` for the lines that apply everywhere.

*Usage:*

"Why is every session starting with 4k tokens already burned?"

"Check `AGENTS.md` — someone pasted the entire style guide in there instead of putting it behind a skill."

## **Progressive disclosure**

Loading only the context an agent needs right now, with context pointers to the rest. Borrowed from UI design, where it means

showing users only the controls relevant to their current task and hiding the rest behind a click.

The technique exists because context is a cost twice over. Every token loaded up front is billed as input tokens on every turn, and every token spends attention budget whether the agent needs it or not. An AGENTS.md stuffed with the full style guide, deployment runbook, and database conventions makes the agent worse at all of them — the instructions that matter for the current task are diluted by the ones that don't. The tell is an agent that ignores rules you know are in its context: they're in there, but buried.

Progressive disclosure inverts this. Keep the always-loaded layer small — a sentence per topic and a pointer to where the detail lives. The agent reads the style guide when it's writing a component, the deployment runbook when it's deploying, and neither when it's fixing a test. Skills are the pattern built into the harness: a short description loaded every session, the full instructions only when triggered.

*Usage:*

"Should I dump the entire style guide into AGENTS.md?"

"No — progressive disclosure. Reference the style guide as a skill the agent loads when it actually needs to write a component.

AGENTS.md pays the token cost every turn."

## **Context pointer**

A mention in one document that points to another, so the agent can pull it into the context window only when the task calls for it. The unit progressive disclosure is built from.

The reason to use a pointer (instead of inlining the content) is cost. A pointer is one line in the context window. The document behind it might be thousands of tokens, but those tokens cost nothing until the agent actually follows the pointer. Inline a 2,000-token runbook in AGENTS.md and every session pays for it; replace it with "deploy

process: see `internal/deploy.md`" and only the sessions that deploy ever load it. The agent follows the pointer with a tool call when the task matches.

A pointer needs two parts to work: a stable path, and enough description for the agent to know when following it is worth it. A bare path is a pointer the agent has no reason to follow; "see `internal/deploy.md`" with no hint of what's inside gets skipped by a session that needed it. Write the line so it matches how tasks present: "release, deploy, or rollback — read `internal/deploy.md` first".

Pointers are everywhere once you look: lines in `AGENTS.md`, skill descriptions (the harness loads the description; the skill body waits behind it), filenames in a directory listing, links between docs.

A pointer can also tie a secondary source back to the primary source it was derived from — the compaction summary that names the original transcript, the doc that names the source file it describes. This makes the secondary source's lossiness recoverable: when the summary turns out not to be enough, the agent follows the pointer and reads the original, instead of working from whatever the summary kept.

*Avoid:* "reference" — too dry; doesn't convey that following it pulls more context in. "Portal" — too florid.

*Usage:*

"`AGENTS.md` is getting huge."

"Most of it should be context pointers, not content. Keep the always-on rules inline; turn the deploy runbook and the style guide into skills and leave a context pointer behind."

## **Skill**

A teachable capability bundled as a unit — instructions and resources for doing one task well, kept in the environment until a

context pointer pulls it into the context window for the task at hand. The unit of progressive disclosure in a harness.

Skills are an open standard, defined at [agentskills.io](https://agentskills.io) — originally developed by Anthropic and since adopted by most major harnesses, so a skill written once works across them. The format is a folder containing:

- A `SKILL.md` file — metadata (a name and description, at minimum) plus the instructions themselves
- Optionally, scripts the agent can run
- Optionally, templates and reference material the instructions point to

Only the name and description sit in context by default. When the agent's task matches, it loads the rest. Until then, the skill takes up almost no room — a sentence or two of tokens, however large its full instructions are.

This distinguishes skills from `AGENTS.md`, which is loaded into every session regardless of the task. A skill is read when a particular kind of work comes up — releasing, scaffolding a new service, writing a migration — and ignored the rest of the time.

*Avoid:* "tool" — a tool is what the agent *calls*; a skill is instructions it *reads*.

*Usage:*

"Where should I put the deploy runbook?"

"As a skill — the agent loads it only when the task involves deploys. In `AGENTS.md` it'd burn tokens on every turn for something we use weekly."

## Subagent

An agent spawned by another agent via a tool call. Runs in its own session with its own context window, and reports a single tool result

back. Distinct from a handoff — the parent specifically expects a return; a handoff has no return path. **Cannot spawn further subagents** — the tree is one level deep. Subagents exist to isolate context, not to compose hierarchies.

The point is to keep noisy work out of the parent's context. A broad search or a long file-reading expedition produces pages of tool results, most of which matter only long enough to find the answer. Run inside the parent and all of it stays in the parent's context for the rest of the session. Run inside a subagent and the noise fills a disposable window instead — only the final report lands in the parent's context. The report is a secondary source: the parent gets the subagent's account of what it found, not the raw results, so anything the report leaves out is invisible to the parent.

Subagents also run concurrently — a parent can fan several out at once over independent pieces of work.

*Usage:*

"The grep results are blowing out my context."

"Spawn a subagent to do the search — it'll burn its own context window on the noise and report back the two file paths you actually need."

# Section 7 — Patterns of Work

## Human-in-the-loop

A working pattern where one or more humans pair with the agent during a session — reviewing, redirecting, or collaborating in real time. The human is present and engaged, not just gating individual actions.

The contrast is with AFK work, where the agent runs unattended and you judge the result afterwards. Human-in-the-loop means catching problems while they're still cheap: you see the agent reach for the wrong file, misread the requirement, or start down a dead end, and you redirect it in one sentence — rather than discovering twenty minutes of confident work built on that mistake. Agents don't reliably know when they're off track; left alone, they tend to push forward rather than stop and ask.

Which pattern fits depends on the work. Well-specified, low-risk, easy-to-verify tasks suit AFK. Tasks that are ambiguous, irreversible, or where you'd struggle to review the finished result — a schema migration, a tricky design decision, anything touching production — suit staying in the loop. The judgement call is essentially: how expensive is a wrong turn, and how late would you catch it?

Some work is in-the-loop by nature, because your reactions are the input. Grilling only works with you there to answer the questions; prototyping only works with you there to react to the artifact.

Staying in the loop costs your attention, which is the scarce resource. Part of getting better with agents is moving more work safely out of the loop — with plans, automated checks, and human review at the end instead of supervision throughout.

*Usage:*

"Run this AFK overnight?"



"No, schema migration — keep it human-in-the-loop. I want to see each step and steer if it picks the wrong column to backfill from."

## **AFK**

Away from keyboard. A working pattern where the user kicks off a session and leaves the agent to run unattended. The throughput multiplier of AI coding — many AFK sessions can run in parallel while you sleep, eat, or work on something else. Usually requires a permissive permission mode plus sandboxing to be safe.

When you're not there, the agent handles ambiguity differently. While you're watching, an ambiguous decision surfaces as a question and you answer it; once you've walked away, the agent picks a default and keeps going, and every later decision builds on that guess. The characteristic failure is coming back to hours of finished, confident work built on a wrong call made in the first ten minutes. The work isn't sloppy — it's coherent, just coherent about the wrong thing.

Since you can't give input during the run, give it before and after instead. Before: resolve the ambiguity up front — a grilling session, a written spec — so there are fewer gaps for the agent to fill alone. During: automated checks and automated review stand in for the attention you're not giving, failing fast on what can be caught mechanically. After: the run ends in something reviewable — a PR, not changes already merged. AFK doesn't remove human review; it defers all of it to the end, which is why what arrives at the end has to be worth reviewing. This is also why AX matters most in AFK runs — with no one watching, the environment is the only support the agent gets.

*Avoid:* "background agent" — centers the machine ("running in the background") rather than the human pattern ("user has walked away"). AFK names the fact that matters: the user isn't watching.

*Usage:*

"I'm running this AFK — three sandboxed agents on the refactor, reviewing the PRs in the morning."

"Bypass permissions?"

"Yeah, read-only filesystem, no network."

## Automated check

A deterministic verification that runs in the environment — tests, type checks, lints, build, pre-commit hooks. Pass/fail, no judgement. The signal an agent can self-correct from without involving anyone else. A flaky test is a broken check, not a non-check; automated checks are deterministic *by design*.

Self-correction works as a loop. The agent makes a change, runs the check as a tool call, and the failure output lands in its context window — a type error with a file and line, a failing assertion with expected and actual values. That's enough for the agent to fix the problem and run the check again, around and around until it passes, with no human in the loop. Determinism is what makes the loop trustworthy: the same code always produces the same verdict, so a pass means something. A flaky check poisons this — the agent "fixes" code that was fine, or retries past a real failure.

This is why good checks are a large part of a codebase's AX. An agent in a repo with strict types, a fast test suite, and a linter catches most of its own mistakes before you see them; an agent in a repo with none of those ships whatever it produces. The difference matters most in AFK runs, where checks are the only verification happening during the run. But a check only catches what it asserts — green checks mean the asserted properties hold, not that the code is right. The judgement-shaped gaps are what automated review and human review are for.

*Avoid:* "feedback loop" / "backpressure" — both lump checks together with review. *Avoid:* "test" — tests are automated checks, but not all automated checks are tests.

*Usage:*

"The agent keeps shipping broken code in the AFK runs."

"What automated checks are wired into the sandbox?"

"Just the unit tests."

"Add typecheck and lint — it'll self-correct from those before the PR ever lands."

## **Automated review**

An agent reviewing another agent's work, often with a different model or system prompt. Non-deterministic: it forms a judgement. Runs anywhere — pre-merge on a PR, post-hoc on commit history, mid-session as a subagent. An LLM-as-judge in CI is automated review, not an automated check; what the assertion *does* decides the category, not where it runs.

The separation from the working agent is what makes it work. Asking the agent that wrote the code to review its own work gets you very little — the session that produced the bug also contains the reasoning that produced it, and the agent reads its own conclusions back as confirmation. A reviewer with a fresh context window has none of that attachment: it sees the diff the way a stranger would, which is what review depends on. A different model or a review-specific system prompt sharpens this further — different blind spots, and a system prompt scoped to what you actually care about (security, API contracts, performance) rather than a vague "look for problems".

It slots between the other review layers. Automated checks are deterministic and catch what can be asserted mechanically; human review is expensive and scales worst. Automated review sits in the middle: it catches judgement-shaped problems — a misleading function name, a missed edge case — at machine cost. Because it's non-deterministic, it can miss things and flag non-issues; treat it as

a filter that raises the floor before a human looks, not a gate that replaces one.

*Avoid:* "AI review" / "agent review" — too vague to distinguish from the working agent itself.

*Usage:*

"We're getting too many bad PRs from the AFK runs."

"Add an automated review step before merge — different model, separate system prompt, scoped to security and contract changes."

## Human review

The user reading the code the agent produced and forming a judgement on it. Reading the diff or the changed files counts; reading the agent's *description* of what it did does not — narration is not the artifact. The description is a secondary source, written by the party being reviewed; the diff is the primary source, and review means reading it.

Agents raise the volume of code produced, so review becomes the bottleneck. One useful idea is layering different review strategies. Automated checks catch the mechanical failures, automated review catches the describable ones, and human review is reserved for what only you can judge — whether the change is the right change, whether the approach fits the codebase, whether this should exist at all.

Review is also cheaper earlier. Reading a plan before work starts, or a small diff mid-flight, takes minutes; excavating a finished branch after an AFK run takes longer. Where you place the review checkpoint is a human-in-the-loop decision, not an afterthought.

*Avoid:* "code review" alone — ambiguous between human and automated.

*Usage:*

"I human-reviewed the AFK output."

"You read the diff or just the summary?"

"Diff. The summary said it deleted dead code — turned out the function was called from a generated file."

## Vibe coding

A working pattern where the user accepts the agent's code without human review. The diff is treated as opaque — what matters is whether the program behaves, not what's inside. Automated review and automated checks may still run; vibe coding is silent on both.

The term comes from Andrej Karpathy, who coined it in early 2025: you "fully give in to the vibes" and "forget that the code even exists" — describe what you want, accept what comes back, and judge it by running it.

Vibe coding trades inspection for speed. Reading diffs is usually the slowest step in agent-driven work, so dropping it removes the main bottleneck. For code whose failures are cheap — prototypes, one-off scripts, internal tools — that's a reasonable trade. The risk scales with the code's lifespan and stakes.

The cost arrives later. Vibe-coded changes accumulate into a codebase nobody has read, and behaviour was the only thing checked — so anything behaviour doesn't surface, like a secret written to logs, a missing edge case, or quietly wrong data handling, ships unseen. The first time someone debugs the system is the first time anyone reads the code. With human review gone, whatever automated verification still runs — tests, types, automated review — is the only gate the code passes through.

*Avoid:* "vibe coding" as a synonym for "low-quality AI coding" — the term names the review stance, not the resulting code.

*Usage:*

"Did you read what it changed in the auth flow?"

"Vibe coded it — login still works, that's all I checked."

"Read the diff before you push, vibing on auth is how secrets leak into logs."

## Design concept

The shared understanding of what's being built, held in common between user and agent but separate from any asset. Brooks' term (*The Design of Design*): the conversation, handoff artifacts, and the code are all assets that try to capture or reach the design concept, but none of them *are* it. Quality of the design concept is felt through the quality of the conversation that built it.

The term names the gap behind a familiar frustration: the agent writes exactly what you asked for and it's still wrong. The usual cause is that you hadn't fully figured out what you wanted. The design concept wasn't finished in your own head — your prompt captured the parts you'd worked out, and was silent on the parts you hadn't. The agent filled those silences with its own assumptions, because there was nothing to align with. Nothing malfunctioned. There was no shared design concept, because there wasn't yet a whole one to share.

You can tell a design concept is shared the same way you can with a colleague: the other party starts answering questions you haven't asked yet the way you would. Until then, the work is conversation — grilling is the deliberate version — and writing a spec too early just captures the misalignment in a more durable asset. The design concept also moves as you learn; assets lag it, which is why a spec faithful to last week's understanding can still mislead this week's session.

*Usage:*

"It's writing exactly what I asked for and it's still wrong."

"You don't share a design concept yet — it's filling gaps with assumptions. Keep talking until cancellation, refunds, and partial

fulfilment all line up between you before you let it write a spec."

## Grilling

A technique for developing a design concept with an agent: the agent interviews the user Socratically, one decision at a time, proposing a recommended answer for each. Slows the rush to a finished plan — no handoff artifact is written until the concept stabilises.

The technique exists because agents fill gaps silently. Asked to write a spec from a two-line prompt, the agent doesn't stop at the decisions you haven't made — it picks defaults and writes them in. The result looks complete, and the guesses are indistinguishable from the choices, so you discover them late: at review, or when the built feature handles an edge case in a way you never chose. Grilling inverts this — instead of guessing, the agent has to ask.

It's a human-in-the-loop technique: your answers are the input. When a question can't be answered in conversation — you'd have to see the thing — switch to prototyping.

*Usage:*

"It went straight to writing the spec and got the cancellation logic wrong."

"Grill it first — make it ask you about partial cancels, refunds, and timing before it commits anything to the doc. Cheaper to resolve in conversation than in code."

## Prototyping

Having the agent build a quick, rough version of something, for when conversation is too low-fidelity and you need a real artifact to talk about.

Grilling resolves design decisions in conversation. Conversation is cheap, but it's low-fidelity: some questions can't be answered in words — how an interaction feels, whether an API shape is

ergonomic in real calling code, whether the layout works at real data sizes. The interview hits a question and your honest answer is "I don't know, I'd have to see it." Past that point the discussion circles. Instead, have the agent build the thing, look at it, and come back to the conversation with an answer.

Agents lower the cost of building, which is what makes this practical. A rough version that used to take a day to mock up now takes minutes, so it's worth doing routinely. It's a human-in-the-loop technique: the prototype is there for you to react to.

You usually don't stop at one look. Iterate with the prototype — react, ask for a change, react again — so each round resolves another decision against the real artifact, at a higher fidelity than conversation allows.

A prototype doesn't have to be all-scrappy. You can build the pieces you're actually evaluating to production quality, so when the decision lands, the component or API you reacted to can transfer into the real codebase. This makes prototyping essential material for the spec to reference.

*Usage:*

"We've spent half an hour arguing about whether the wizard should be one page or three steps."

"Words won't settle it — have the agent prototype both. We'll click through them and know in five minutes."

## **DX**

Developer experience — how easy a codebase and its toolchain make it for humans to do good work. Good DX is fast feedback, clear error messages, documentation that answers the question you actually have, and setup that works on the first try. The term long predates AI coding; it's in this dictionary mainly as the contrast for AX.



DX is the interaction between the human and the codebase — nothing more. The main difference between the two audiences is that humans are stateful and agents are stateless. A human learns the codebase once and carries that knowledge into every day after, which is why poor DX is survivable: they route around slow CI by batching their pushes, around missing docs by asking in Slack once, around confusing structure by remembering where things live. The workarounds accumulate, and a team ends up productive in a codebase that fights them.

Agents face the same codebase with none of that accumulation. Stateless across sessions, an agent re-learns the codebase from scratch every time — it benefits from the fast test suite and the clear error messages, but anything it figured out yesterday is gone unless it was written into the environment, which the agent only perceives through tool results. That's the gap AX names: the parts of DX that survive when the developer is an agent, plus concerns humans don't have, like keeping the context window free.

The overlap means DX investment often improves AX for free — strict types, fast tests, and predictable structure help both. The divergence means it doesn't always: a beautiful onboarding doc helps a human for a week and an agent not at all unless it's reachable from AGENTS.md.

*Usage:*

"Our DX is fine — new hires are productive in a week."

"Productive because someone sits with them for that week. The agent doesn't get that week; check the AX separately."

## **AX**

Agent experience — how well the environment is set up for an agent to do good work in a codebase. The agent-facing counterpart to DX. When the same agent performs well in one repo and badly in another — same model, same harness — the difference is usually AX.

The instinct is to blame the model or rewrite the prompt; the fix is more often in the repo.

Good AX has three main dimensions:

| Dimension        | What good AX looks like                                                                                                                                        |
|------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Automated checks | Fast, deterministic automated checks — types, tests, lints — that the agent can self-correct from without a human                                              |
| Architecture     | A codebase the agent can navigate without reading everything: predictable structure, a lot of behaviour behind small interfaces, names that say what things do |
| Free context     | AGENTS.md, skills, and tools kept lean, so most of the context window is available for the task and the agent stays in the smart zone instead of drowning      |

AX and DX overlap — good checks and clean architecture help both audiences — but they diverge. Humans tolerate tribal knowledge, slow CI, and "ask Sarah about the billing module"; agents can't. Agents don't benefit from IDE tooltips or pretty dashboards; they need failures as text in a tool result. A codebase can have good DX and poor AX.

*Avoid:* treating AX as a synonym for DX — the audiences need different investments.

*Usage:*

"The agent writes great code in the API repo and garbage in the frontend."

"The API repo has strict types and a fast test suite; the frontend has neither and forty always-loaded skills. That's an AX gap, not a model problem."